

SOFTWARE ENGINEERING LAB 4

Space Invaders: Debugging & Feature Implementation

Name: Samrudh Patil | SRN: PES1UG24CS417

1. Deliverable Links

Item	Link
Before Video (10s Bug Clip)	https://drive.google.com/file/d/1R_IrPNGwi9eg_aFYdjUfJbezwhTbwlVu/view?usp=sharing
After Video (10s Working Clip)	https://drive.google.com/file/d/1jJZSjnTkjWDD3AQW97pUZ67wQdE-Kwy0/view?usp=sharing
Assigned Starter Repository	https://github.com/SETAPESU26/46_space-invaders
Individual GitHub Repository	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417

2. Commit History (Lab-4 Folder)

Each required task was committed separately to the repository:

Commit	Task	Message
5e2575a	Task 1	Task 1: Fix bullet collision detection via shallow copy iteration
632e242	Task 2	Task 2: Implement graphical Game Over screen and final score display
ff39973	Task 3	Task 3: Add replay options with Easy, Medium, and Hard difficulty presets
ee8e464	Task 4	Task 4: Add retro 8-bit sound effects for shooting, enemy destroy, and game over

3. Tasks Completed & Code Snippets

Task 1: Bullet Collision Detection Fix

Bug: Bullets passed straight through enemies when shooting rapidly. The code was iterating over `self.player_bullets` while calling `self.player_bullets.remove(bullet)` inside the same loop. Modifying the list during iteration shifted index positions, skipping the next bullet.

Fix: Iterate over a shallow copy of the list using `self.player_bullets[:]` so removing items from the original list does not cause skipped bullets.

```
# Iterate over shallow copy to avoid skipping elements
for bullet in self.player_bullets[:]:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
```

```
self.sound_manager.play_enemy_hit()  
break
```

Task 2: Graphical Game Over Screen

Bug/Limitation: In the starter code, when the player was hit or enemies reached the bottom, the window simply froze and printed the score to the console.

Fix: Added a graphical Game Over overlay rendered directly on the screen with a dark translucent background, GAME OVER / VICTORY text, final score display, and user input handling.

```
if self.game_over:
    # Semi-transparent overlay
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 195))
    screen.blit(overlay, (0, 0))

    # Centered modal box
    card_rect = pygame.Rect(box_x, box_y, box_width, box_height)
    pygame.draw.rect(screen, (24, 24, 34), card_rect, border_radius=14)
    pygame.draw.rect(screen, accent_color, card_rect, width=3, border_radius=14)

    # Title & Final Score
    title_surf = self.title_font.render(title_text, True, accent_color)
    screen.blit(title_surf, title_surf.get_rect(center=(self.width // 2, box_y + 45)))
    score_surf = self.font.render(f"Final Score: {self.score}", True, WHITE)
    screen.blit(score_surf, score_surf.get_rect(center=(self.width // 2, box_y + 95)))
```

Task 3: Replay Option with Difficulty Settings

Fix: Added 3 difficulty levels (Easy, Medium, Hard) that adjust enemy speed and fire rate. Added a difficulty badge on the HUD, and keys 1 (Easy), 2 (Medium), 3 (Hard) to restart on Game Over, or Q to exit.

```
DIFFICULTIES = {
    "Easy": {"enemy_speed": 1.0, "fire_chance": 0.001, "color": (100, 220, 100)},
    "Medium": {"enemy_speed": 1.6, "fire_chance": 0.003, "color": (240, 200, 60)},
    "Hard": {"enemy_speed": 2.5, "fire_chance": 0.007, "color": (255, 80, 80)},
}

def reset_game(self, difficulty=None):
    if difficulty and difficulty in self.DIFFICULTIES:
        self.current_difficulty = difficulty
        diff_cfg = self.DIFFICULTIES[self.current_difficulty]

    self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
    self.enemy_grid = EnemyGrid(self.width, speed=diff_cfg["enemy_speed"])
    self.enemy_fire_chance = diff_cfg["fire_chance"]
    self.player_bullets.clear()
    self.enemy_bullets.clear()
    self.score = 0
    self.game_over = False
```

Task 4: Sound Effects

Fix: Used Python's wave and math modules to synthesize 8-bit retro arcade sound effects without requiring external sound files:

- **Player shoot:** Laser frequency sweep (shoot.wav)
- **Enemy hit:** Explosion noise (enemy_hit.wav)
- **Game over:** Descending arpeggio sequence (game_over.wav)

```
class SoundManager:
    def __init__(self):
        self.shoot_sound = pygame.mixer.Sound("game/assets/sounds/shoot.wav")
        self.hit_sound = pygame.mixer.Sound("game/assets/sounds/enemy_hit.wav")
        self.game_over_sound = pygame.mixer.Sound("game/assets/sounds/game_over.wav")

    def play_shoot(self): self.shoot_sound.play()
    def play_enemy_hit(self): self.hit_sound.play()
    def play_game_over(self): self.game_over_sound.play()
```

4. Prompts & AI Pair Programming Summary

- **Prompt 1:** Investigated why bullets passed through enemies without hitting them. AI explained in-place list modification during iteration and suggested `self.player_bullets[:]`.
- **Prompt 2:** Created an automated simulation to reproduce and record the 10-second before-video showing the skipped bullet collision bug.
- **Prompt 3:** Designed and implemented the graphical Game Over screen, replacing the terminal print statement with an in-game screen overlay.
- **Prompt 4:** Added difficulty options (Easy, Medium, Hard) and replay functionality.
- **Prompt 5:** Generated retro sound effects using standard library wave module and connected sound playback to player firing, enemy destruction, and game over.